Введение в синтаксис

Почему возник Go, рефлексия

- В 2007 году в Google накопилась реальная инженерная боль:
  - C++ - огромное время компиляции, опасная работа с памятью
  - Java - многословие, JVM overhead, сложная модель конкурентности
  - Python - медленный, динамическая типизация скрывает ошибки до рантайма

- Публичный анонс - 10 ноября 2009 года. Go 1.0 - март 2012.
- С тех пор - полная обратная совместимость.
- Текущая стабильная версия - Go 1.26

Три принципа Go

- Простота - это дизайн-решение, не упущение. Нет наследования. Нет перегрузки операторов. Нет исключений. Нет неявных преобразований типов. Каждое «нет» - осознанный выбор в пользу читаемости и предсказуемости.
- Производительность через предсказуемость. Компиляция в нативный код. Нет виртуальной машины. Программист должен понимать, что происходит в памяти.
- Конкурентность - примитив языка, не библиотека. Горутины и каналы встроены в язык, а не добавлены поверх.

Установка Go

Microsoft Windows	Apple macOS (ARM64)	Apple macOS (x86-64)	Linux	Source
Windows 10 or later, Intel 64-bit processor	macOS 12 or later, Apple 64-bit processor	macOS 12 or later, Intel 64-bit processor	Linux 3.2 or later, Intel 64-bit processor	
go1.26.1.windows-amd64.msi	go1.26.1.darwin-arm64.pkg	go1.26.1.darwin-amd64.pkg	go1.26.1.linux-amd64.tar.gz	go1.26.1.src.tar.gz

- Скачать
  - https://go.dev/doc/install

```
> go version
go version go1.26.1 windows/amd64
```

```
> go env GOPATH    # рабочая директория Go
> go env GOROOT    # где установлен Go сам
```

CLI

```
# Запуск и сборка
go run main.go          # компилирует и запускает (без сохранения бинарника)
go build .              # компилирует в бинарник (имя = имя модуля)
go build -o myapp .     # компилирует с заданным именем
go install .            # компилирует и кладёт бинарник в GOBIN

# Форматирование и анализ
go fmt ./...            # форматирует все .go файлы в проекте
go vet ./...            # статический анализ, находит типичные ошибки

# Тесты
go test ./...           # запустить все тесты
go test -v ./...        # с подробным выводом
go test -race ./...     # детектор гонок данных
```

CLI

```
# Модули
go mod init <имя>        # создать go.mod
go mod tidy              # синхронизировать зависимости
go get <pkg>             # добавить зависимость
go get -tool <pkg>       # добавить tool-зависимость

# Документация и инспекция
go doc fmt.Println       # документация функции
go doc -src fmt.Println # показать исходный код функции
go env                   # все переменные окружения Go
go tool <name>           # запустить встроенный инструмент
```

Кросс-компиляция

```
# Собрать Linux-бинарник на macOS
GOOS=linux GOARCH=amd64 go build -o myapp-linux .

# Уменьшить размер бинарника (убрать debug-символы):
go build -ldflags="-s -w" -o myapp .

# Все поддерживаемые платформы:
go tool dist list
# android/arm, darwin/amd64, linux/arm64, windows/amd64, ...
```

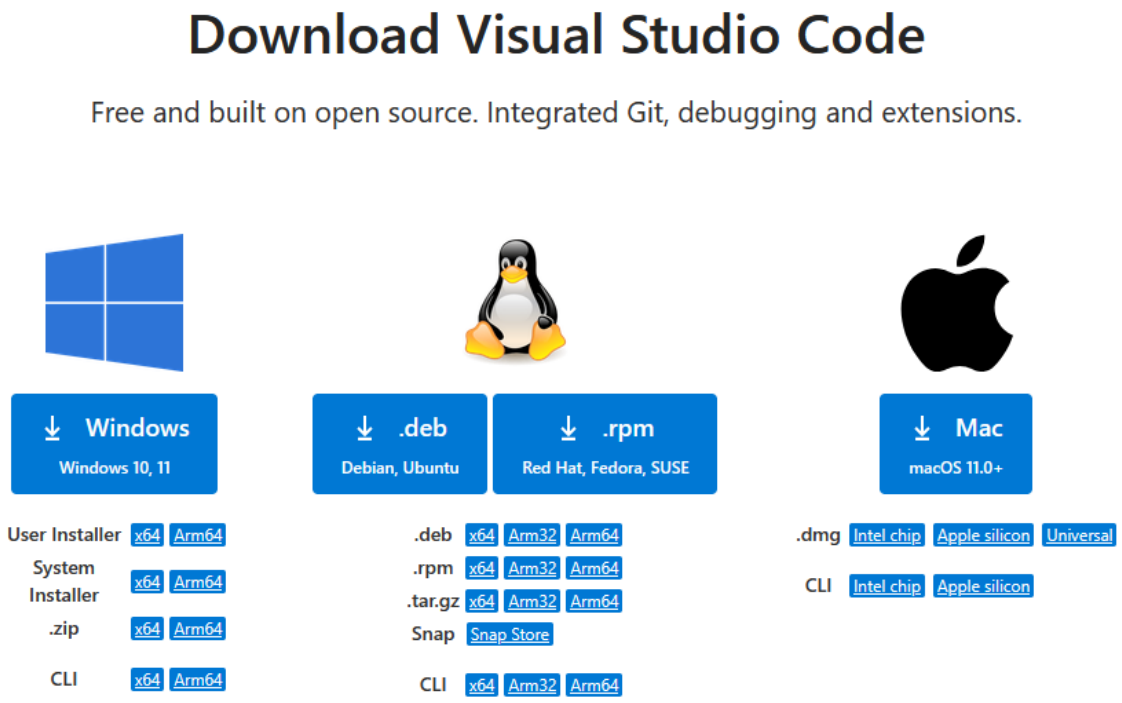
CLI и env (go env)

```
GOVERSION=go1.26.1
GOROOT=C:\Program Files\Go
GOPATH=C:\Go
GOBIN=""   # если пусто - используется GOPATH/bin
GOOS=windows  # целевая ОС
GOARCH="amd64"  # целевая архитектура
GOTOOLCHAIN="auto" # управление версией тулчейна

# Телеметрия:
> go telemetry        # показать режим
> go telemetry on     # включить отправку на telemetry.go.dev
> go telemetry off    # выключить (данные остаются локально)
```

Редакторы

Visual Studio Code https://code.visualstudio.com/Download

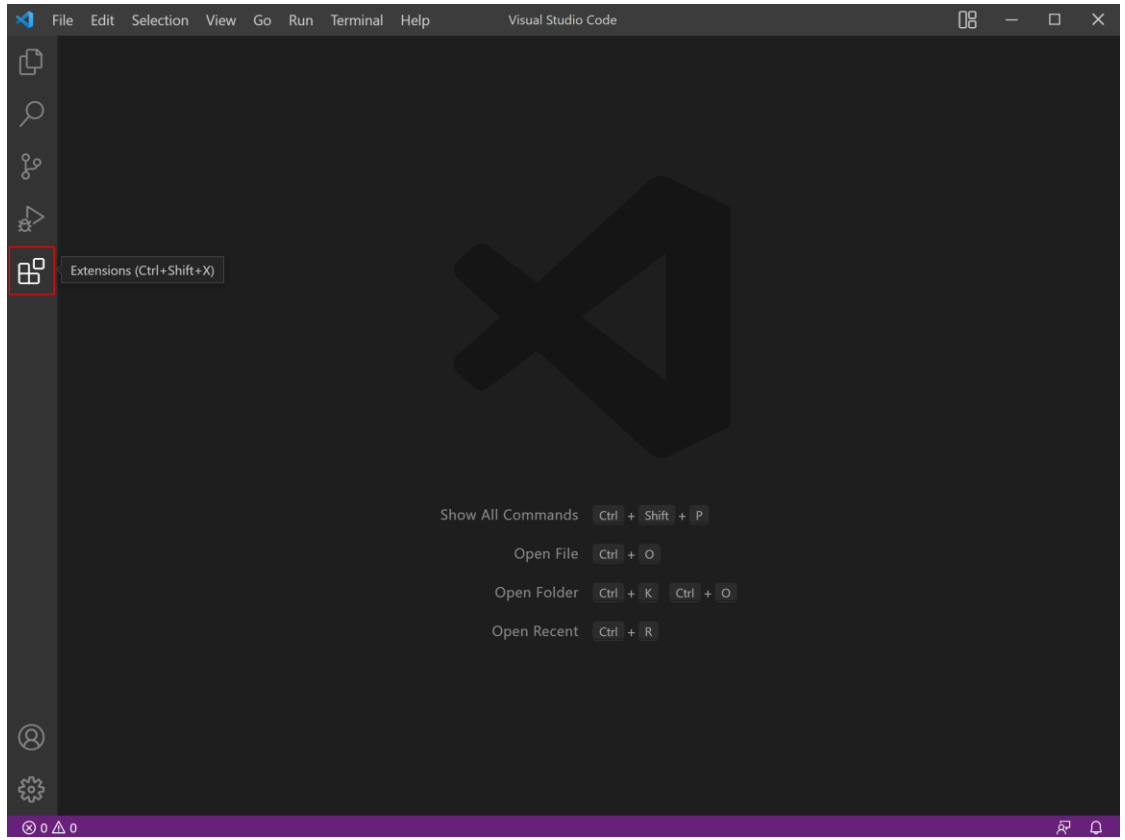


Редакторы

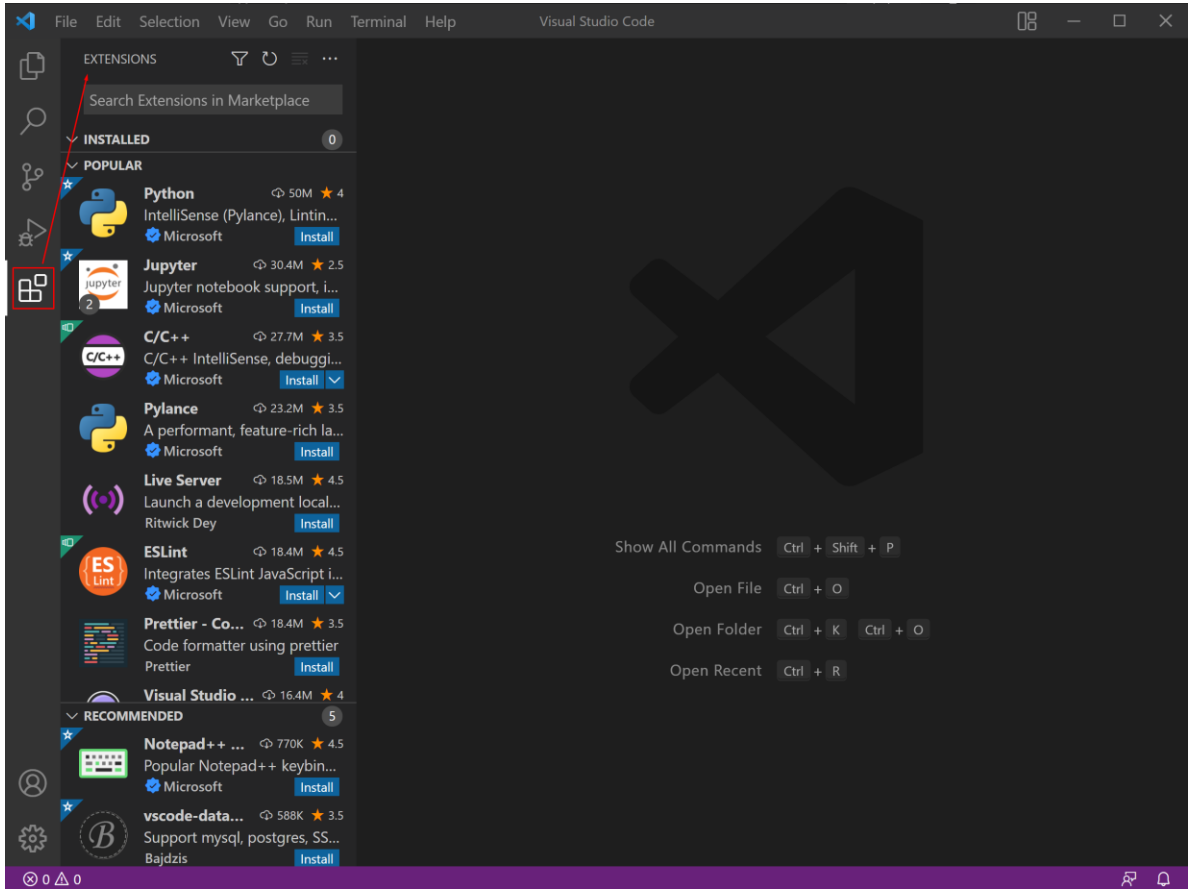
JET BRAINS

GoLand

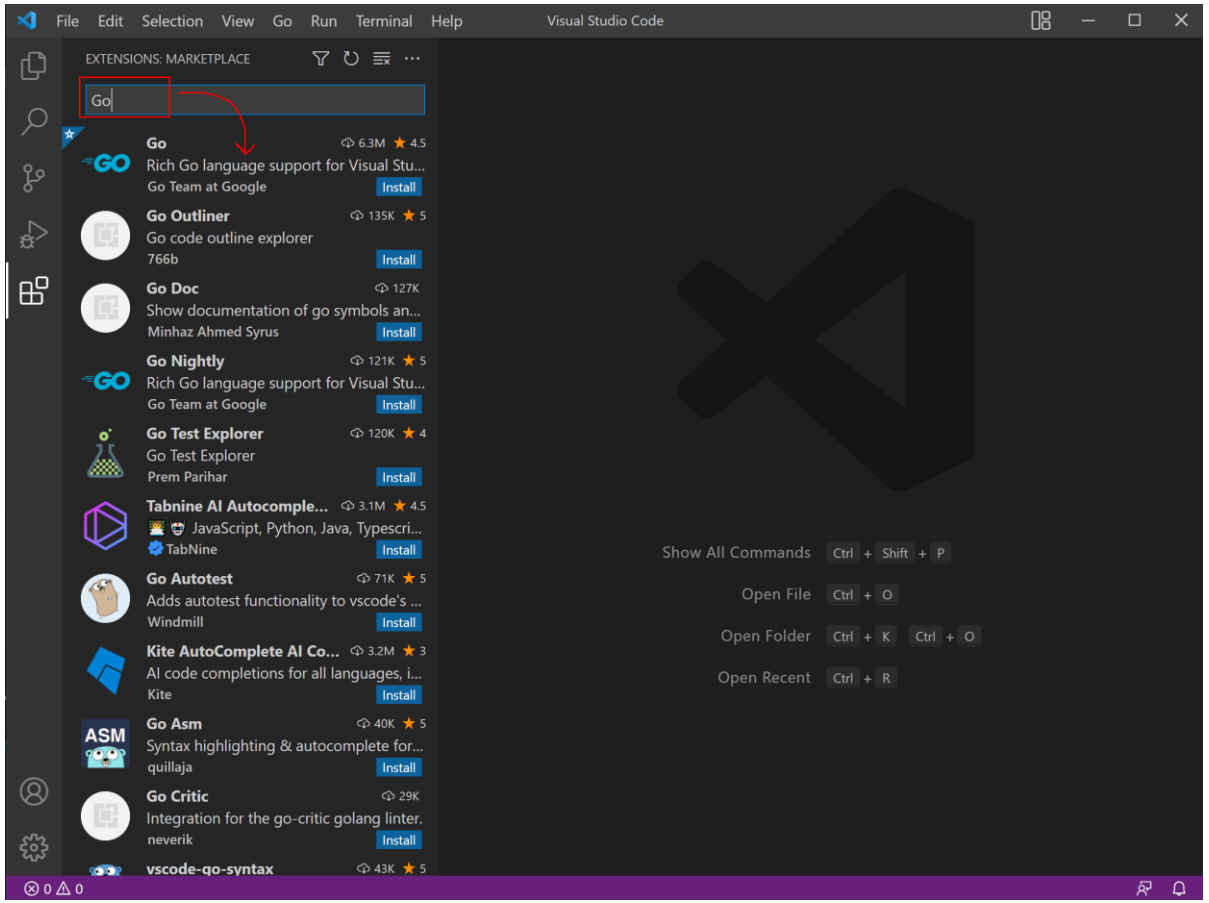
VS Code установка расширений для Go



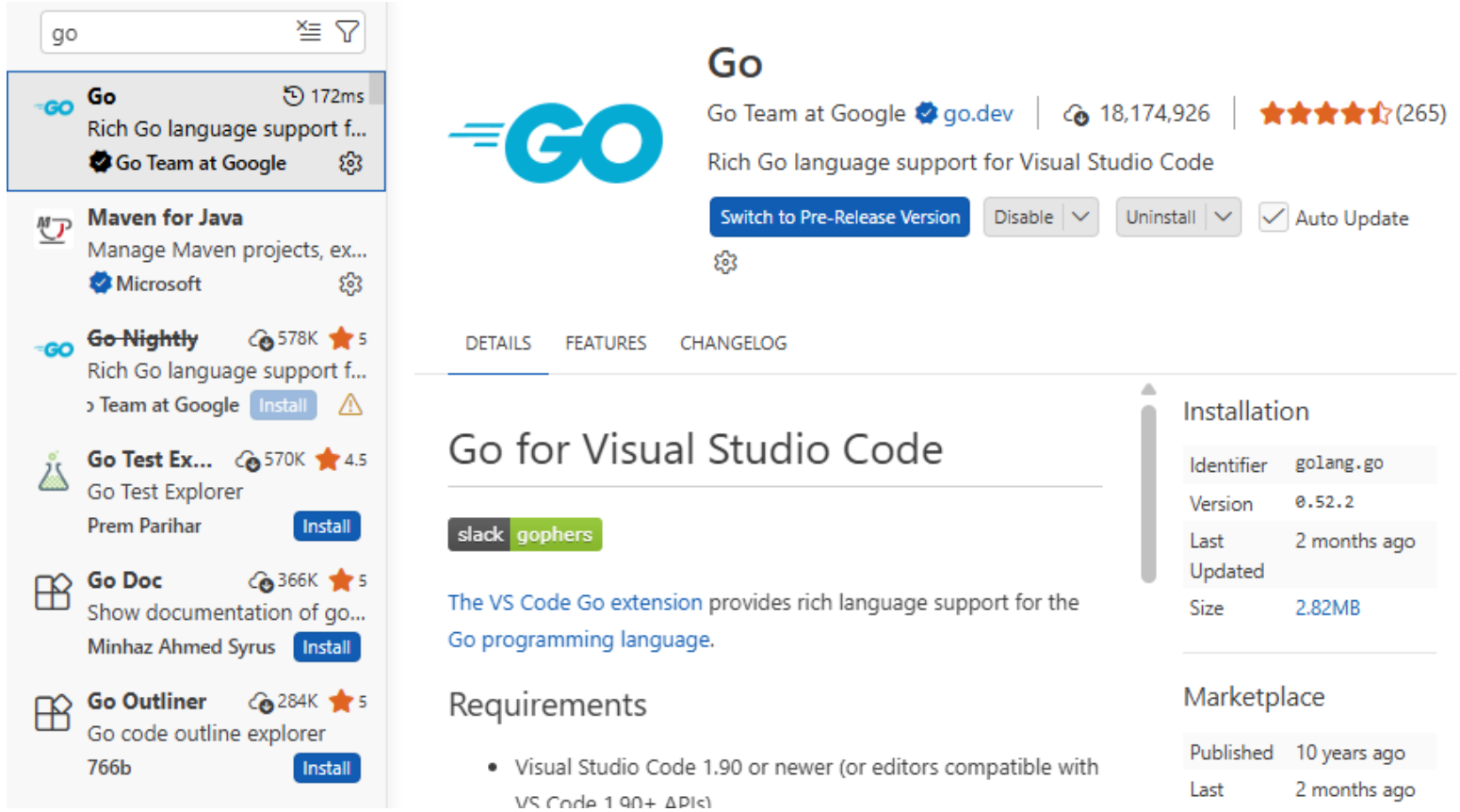
VS Code установка расширений для Go



VS Code установка расширений для Go

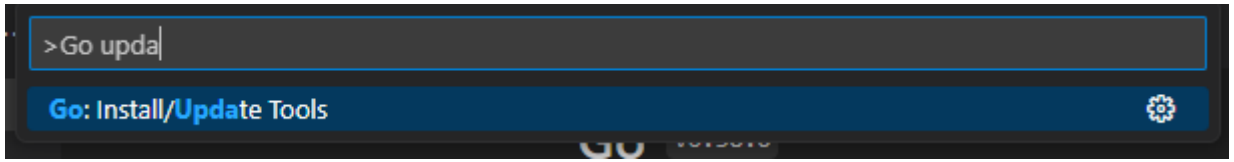


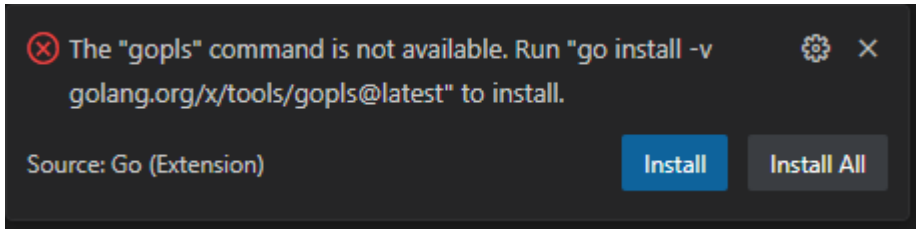
VS Code установка расширений для Go

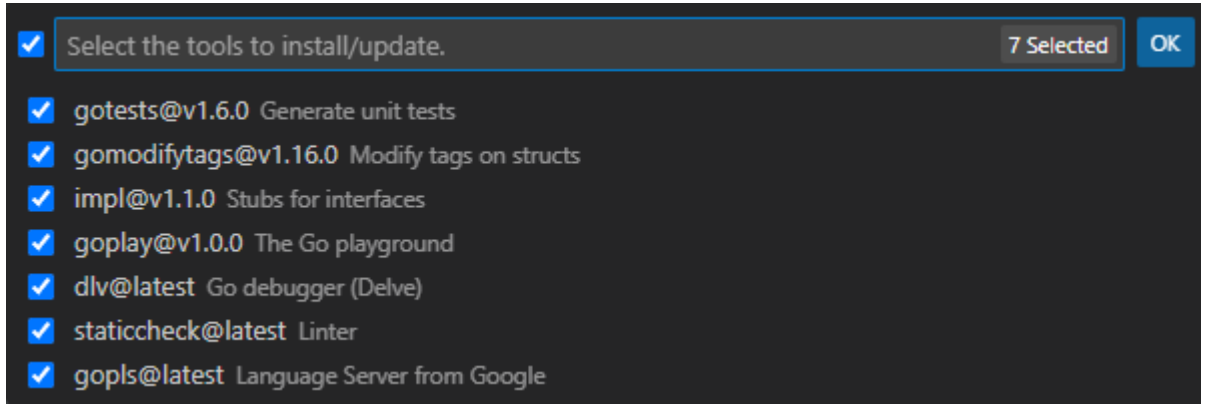


VS Code установка расширений для Go

- CTRL + Shift + P



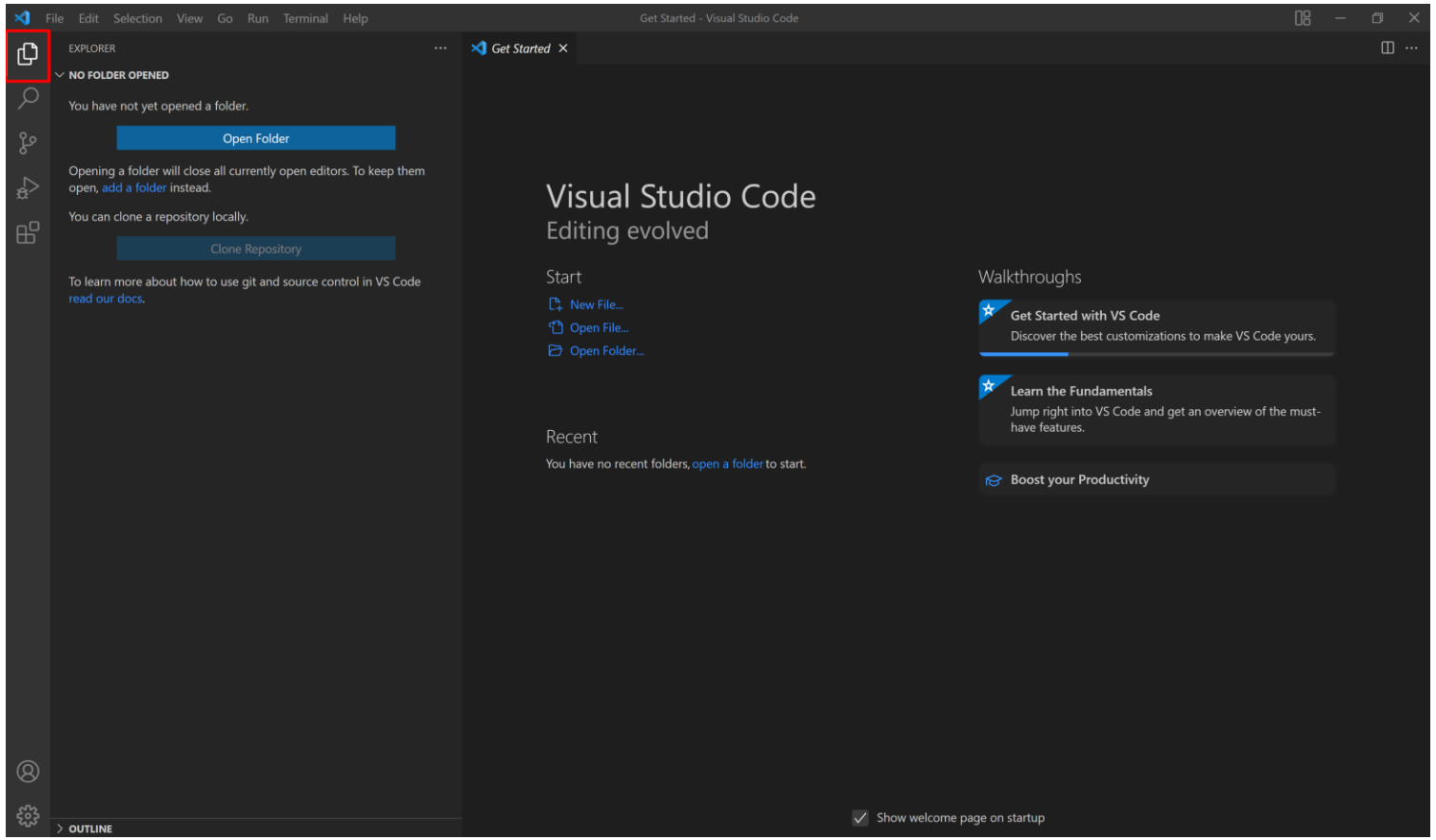




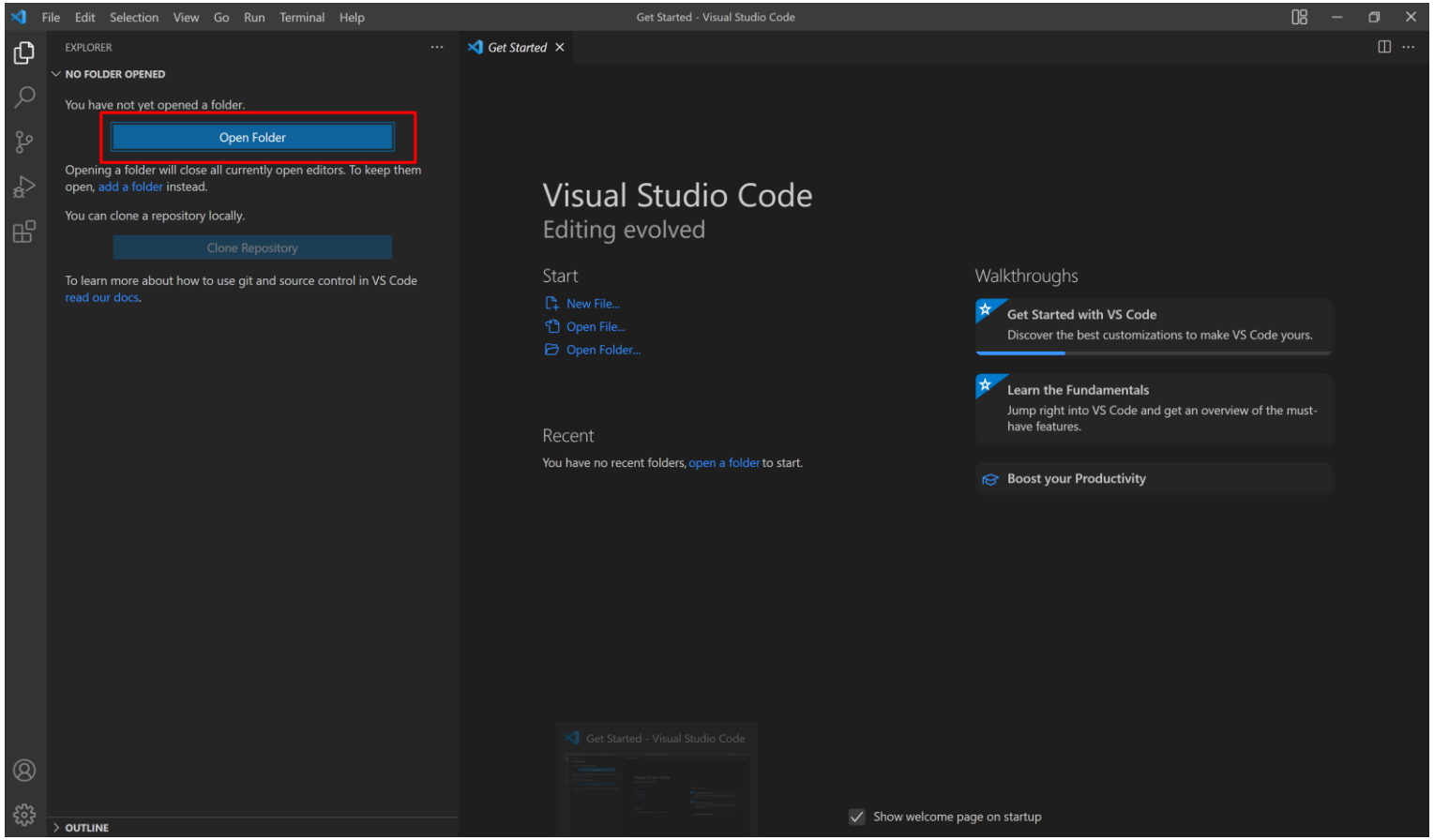
Что устанавливается

- gopls
  - Language Server (автодополнение, рефакторинг, навигация, форматирование)
- dlv
  - Отладчик Delve
- staticcheck
  - Расширенный линтер
- gotest
  - Запуск тестов из редактора
- impl
  - Генерация методов интерфейса

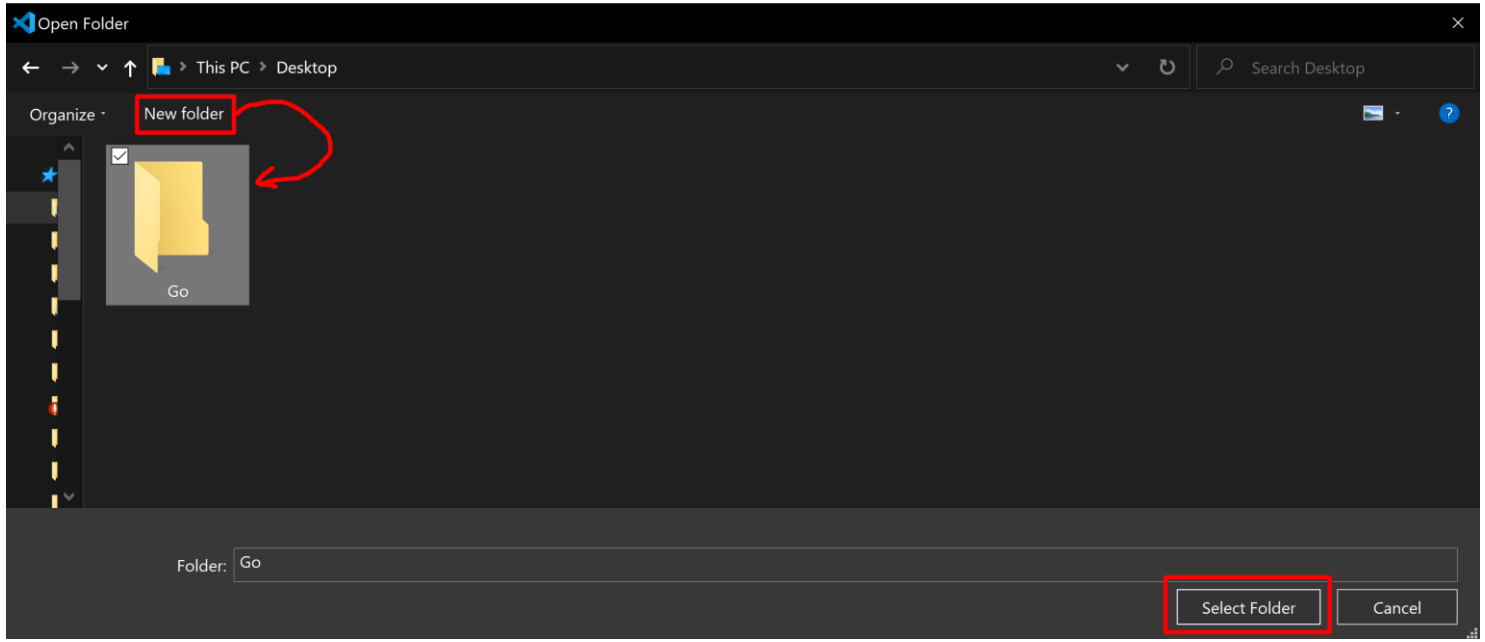
Подготовьте папку



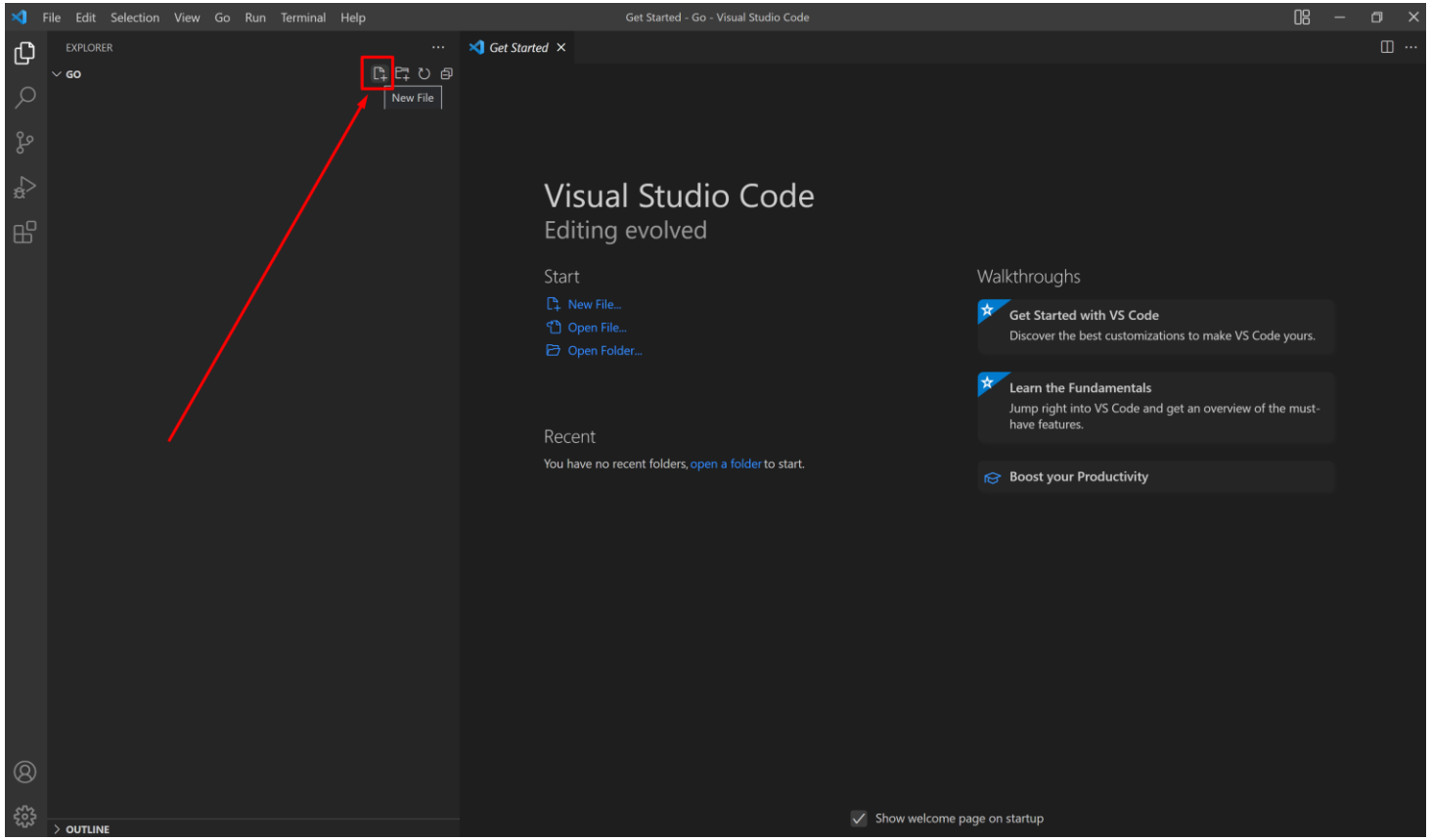
Подготовьте папку



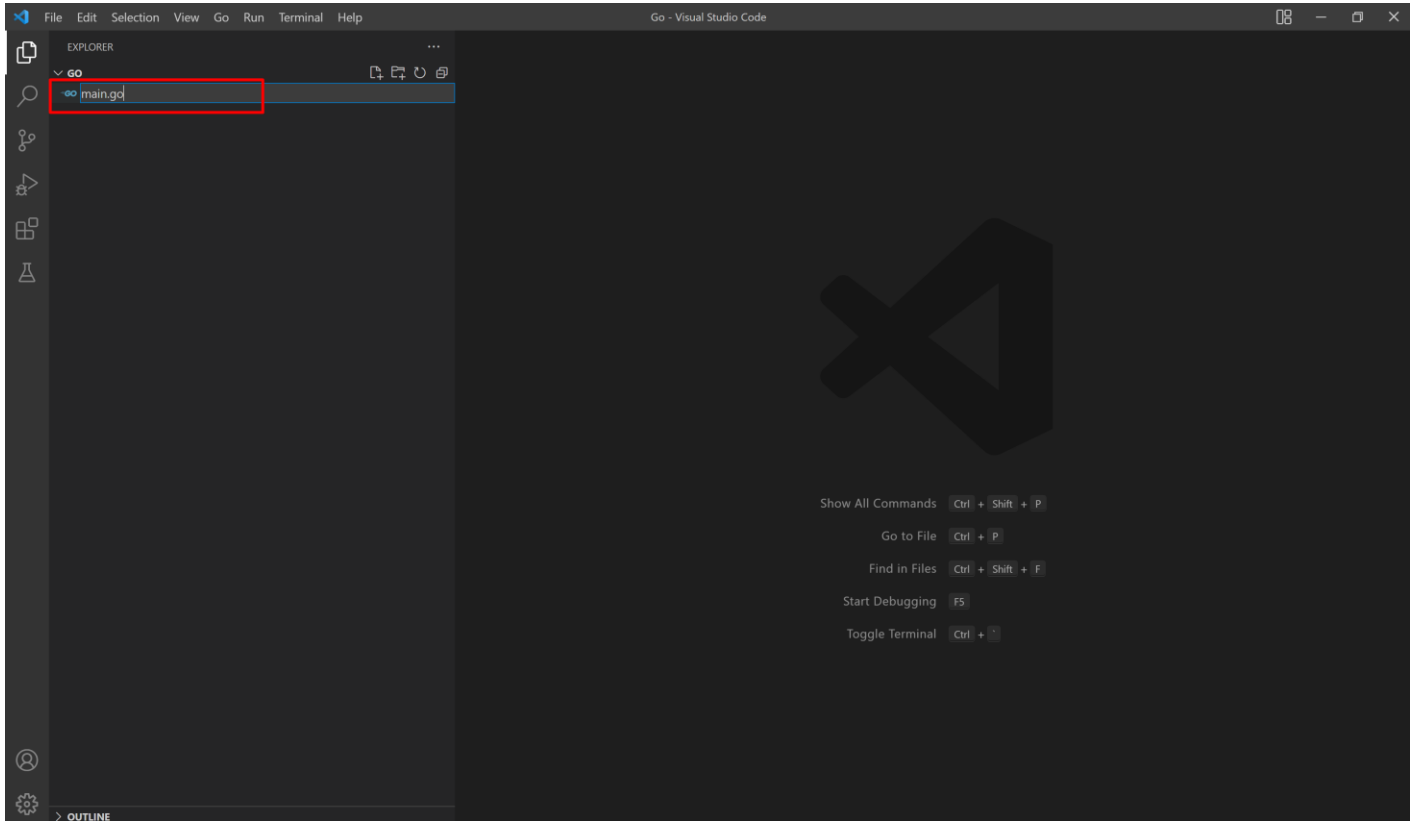
Подготовьте папку



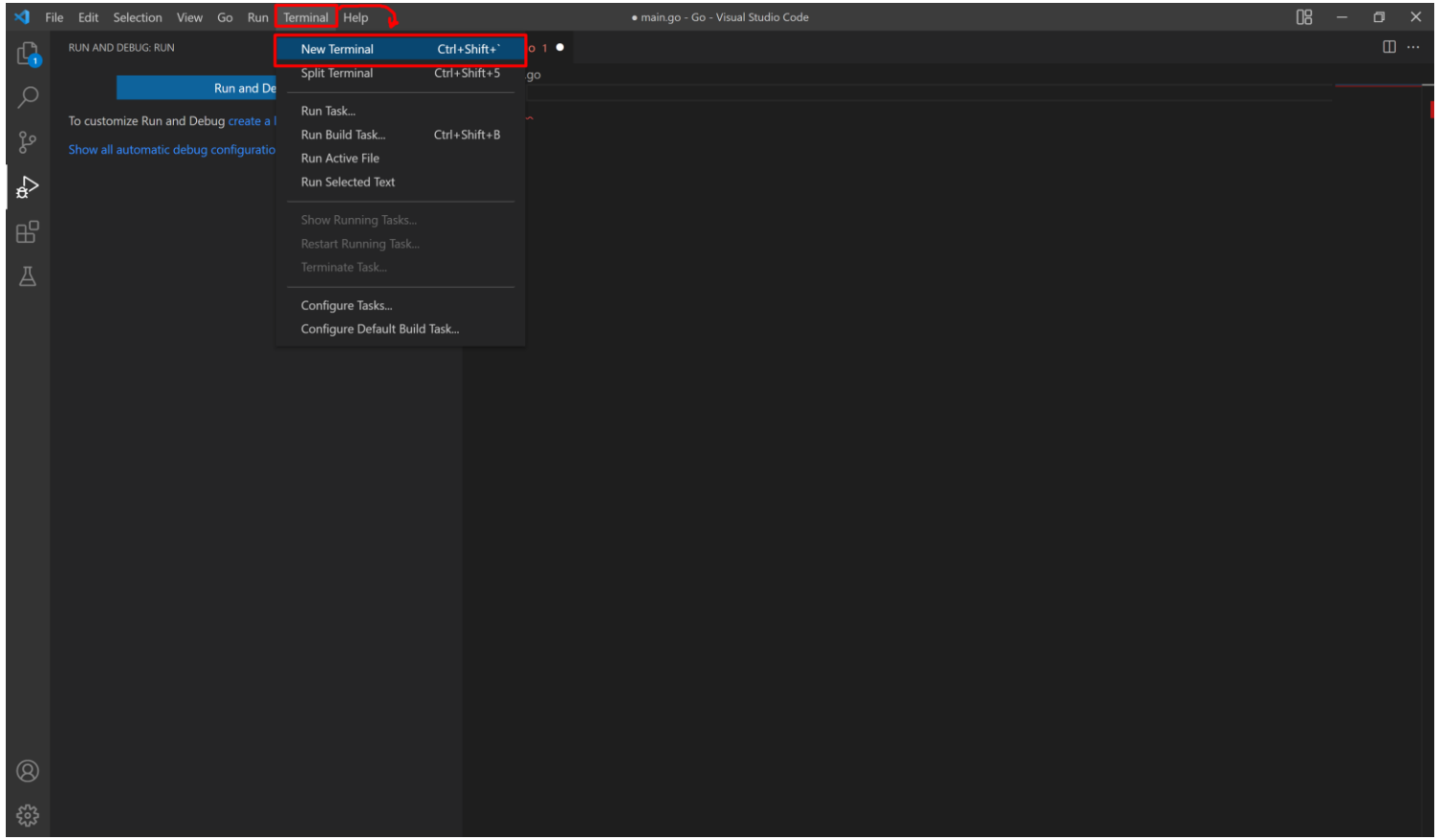
Создайте файл



Создайте файл



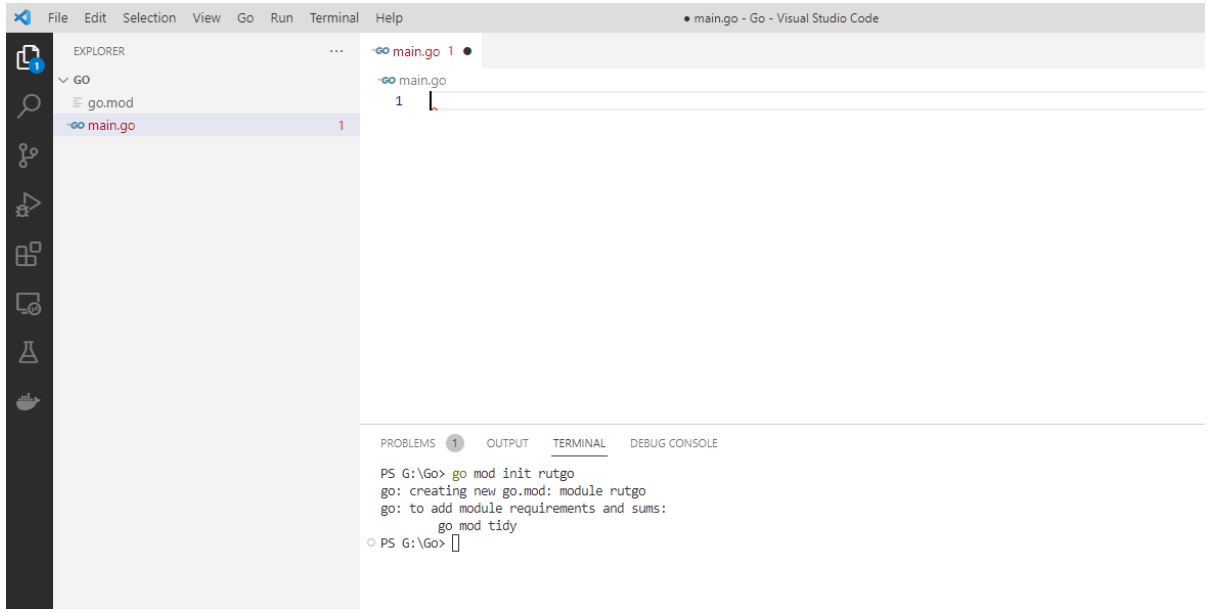
Терминал


File Edit Selection View Go Run Terminal Help
• main.go - Go - Visual Studio Code
RUN AND DEBUG: RUN
Run and De
To customize Run and Debug create a l
Show all automatic debug configuratio
New Terminal Ctrl+Shift+`
Split Terminal Ctrl+Shift+5
Run Task...
Run Build Task... Ctrl+Shift+B
Run Active File
Run Selected Text
Show Running Tasks...
Restart Running Task...
Terminate Task...
Configure Tasks...
Configure Default Build Task...

Терминал

- Введите

```
go mod init {имя} # demos
```



go mod и версия языка

- С Go 1.11 каждый проект - независимый модуль с go.mod

go.mod

go.mod

Reset go.mod diagnostics | Run govulncheck | Run go mod tidy

```
1 module demos
2
3 go 1.26.1
4
```

CLI

```
> go mod tidy    # добавить недостающие, удалить лишние зависимости
> go mod graph   # дерево зависимостей
> go mod vendor  # скопировать зависимости в ./vendor
```

DEMO

package

- Каждый .go файл обязан начинаться с package <имя>

- Все файлы в одной директории - один пакет
- Имя пакета ≠ путь импорта (но обычно совпадает с именем директории)
- Только package main + func main() = исполняемая программа
- Всё остальное - библиотека

package

```
myproject/
├── main.go          package main
├── math/
│   └── calc.go      package math    # импортируется как myproject/math
└── http/
    └── server.go    package http
```

import

```go
// Одиночный импорт
import "fmt"
// Групповой импорт
import (
    "fmt"
    "math"
    "strings"
    "strconv"
)
// Импорт с псевдонимом
import (
    f "fmt"          // f.Println("hello")
    m "math"         // m.Sqrt(4)
)
```

import

```go
// Импорт только ради побочных эффектов (init())
import _ "image/png"

// Импорт стандартной библиотеки vs внешних пакетов
import (
    "fmt"                                   // stdlib
    "net/http"                              // stdlib, вложенный пакет
    "math/rand/v2"                          // stdlib v2 (Go 1.22+)

    "github.com/gin-gonic/gin"              // внешний пакет
    "github.com/username/myproject/utils"   // ваш пакет
)
```

Неиспользованный импорт - ошибка компиляции

Экспортируемость

- В Go нет ключевых слов public / private.
- Правило одно - заглавная буква = экспортируется (видна вне пакета).

```
package mypackage

// Экспортируемые - видны из других пакетов
var MaxSize = 100
func Calculate() int { return 42 }
type Config struct { ... }

// Неэкспортируемые - только внутри пакета
var internalCache = map[string]string{}
func validateInput(s string) bool { ... }
type internalState struct { ... }
```

Экспортируемость

- В Go нет ключевых слов public / private.
- Правило одно - заглавная буква = экспортируется (видна вне пакета).

```
// Из другого пакета:
import "mypackage"

mypackage.Calculate()
mypackage.MaxSize
mypackage.validateInput() // unexported
```

Комментарии и godoc

```
// Однострочный комментарий

/*
    Многострочный комментарий
    ☺
*/

// Godoc-комментарий должен начинаеться с имени идентификатора, так принято.
// Calculate возвращает сумму двух целых чисел.
func Calculate(a, b int) int {
    return a + b
}
```

Комментарии и godoc

```
> go doc calc.Calculate
package calc // import "demos/calc"

func Calculate(a, b int) int
    Godoc-комментарий должен начинаться с имени идентификатора, так принято.
    Calculate возвращает сумму двух целых чисел.
```

DEMO с print

Запуск

```
go run main.go
go run .              # все файлы в текущей директории

go build .            # создаст ./myproject (или myproject.exe на Windows)
go build -o app .     # явное имя бинарника

# Запуск бинарника:
./app                 # Linux/macOS
.\app.exe             # Windows
```

DEMO с nulls

Таблица целых типов

Тип	Размер	Диапазон	Применение
`uint8`	1 байт	0 … 255	байты, RGB-каналы
`uint16`	2 байта	0 … 65 535	порты, Unicode BMP
`uint32`	4 байта	0 … 4 294 967 295	IPv4-адреса
`uint64`	8 байт	0 … 18 446 744 073 709 551 615	timestamps, ID
`int8`	1 байт	−128 … 127	редко
`int16`	2 байта	−32 768 … 32 767	аудиосэмплы
`int32`	4 байта	−2 147 483 648 … 2 147 483 647	Unix timestamp
`int64`	8 байт	-9.2×10^{18} … 9.2×10^{18}	общее использование
`int`	4 или 8	зависит от платформы	индексы, счётчики
`uint`	4 или 8	зависит от платформы	редко
`uintptr`	4 или 8	зависит от платформы	unsafe-арифметика
`byte`	1 байт	= uint8	байты, ASCII
`rune`	4 байта	= int32	Unicode code points

DEMOS

Будут вопросы? Не стесняйтесь!